

Writing a note

NSDF-Data student guide 3 — writing a note, for every computer

Version 5 · 5 October 2026

Tip

This is guide 3 of 5. It is about *writing* the note. Guide 4 covers *publishing* it, and guide 5 covers publishing with an AI assistant.

Tip

What a note is. A note is a short, honest write-up of one question you investigated: what you asked, what you did, what you found, and how sure you are. Notes are how the lab keeps a record that someone else can follow and check. You do not need to be an expert to write one. You do need to be specific.

Before you start

- You have finished the setup guide and the first analysis for your computer.
- You know which question you are answering. If you do not, pick a small one from the project's *Next steps* sections (for example, a number you can reproduce from a note).
- You have a notebook (`.ipynb`) or a script that produces the numbers and figures you will show. Keep it. The note points to it.

What a note is for

A reader should be able to answer four questions after reading your note:

1. **What was the question?**
2. **What data and code did you use?**
3. **What did you find, and how sure are you?**
4. **What would you do next?**

If your note does not answer these, it is not finished, however long it is.

The parts of a note

Only the header is required. Every other section is a suggestion. Use the sections that help your reader, and leave out the ones that do not apply. A short note with a header and one clear result is complete. A long note with no caveats is not.

The table below lists the sections most notes use, in a sensible order. Each one you use becomes a heading, with an outline link.

Section	What goes in it	Length
Title	The question, as a sentence. “Why does X happen in Y?” is better than “Analysis of X.”	one line
Header	Author, date, status, and one-sentence description (see below).	a few lines
Code repository	The notebook and library files you used, each pinned to the commit that produced the results.	a short list
Data and setup	Which series, dump and detector; which quantities; which windows and cuts. Define any word a newcomer would not know.	one or two paragraphs
The question	The question again, and why it matters.	a paragraph
Results	Your figures and tables, each with a caption that says what the reader is looking at.	as needed
Reading	What you think the results mean, and what they do <i>not</i> show.	a few bullets
Caveats	What could be wrong. Small sample? One channel only? A cut you chose by eye?	a short list
Next steps	Concrete things to do next.	a short list

Tip

Caveats are strongly suggested. A note that states its limits is trusted more than one that does not, however good the plots look. If you leave caveats out, be sure there is nothing you would want a reader to know about your result.

Which file to write in

Write your note in a **plain text file** with the extension `.md`. The name matters: `my-note.md`, not `my-note.md.txt` and not `my-note.docx`.

Option	Use it when	What to do
GitHub web editor (any computer)	You want no install at all	Open the file in the repository on github.com and click the pencil icon. Choose a new file named <code>notes/src/your-note.md</code> .
VS Code (any computer)	You want a comfortable editor	Install VS Code, open the NSDF-Data folder, click New File , and name it <code>your-note.md</code> .
Notepad (Windows)	You only want something simple	Open Notepad, write, then File, Save As . Set <i>Save as type</i> to All Files and name the file <code>your-note.md</code> . In Notepad, leave the encoding as UTF-8 .
TextEdit (macOS)	You only want something simple	Open TextEdit, choose Format , then Make Plain Text before typing. Save as <code>your-note.md</code> . If it offers <code>.txt</code> , pick <i>Use .md</i> when asked.
gedit or Text Editor (Linux)	You use a standard desktop	Open the editor, write, and save as <code>your-note.md</code> .

Careful

Do not write your note in Word or Google Docs. Word adds hidden formatting, turns straight quotes into curly ones, and changes characters in code. Those changes break the page and the code you paste in. If you already wrote in Word, convert it (see the next section).

To see the full file name on Windows, open File Explorer, then **View**, then tick **File name extensions**. Check that the name ends in `.md`.

Converting a Word or Google Docs file

If your draft is in Word or Google Docs, convert it to Markdown with a program called **pandoc**. It is free and runs from the terminal you already know from the setup guide.

Step 1. Export your draft. - In **Word**: **File**, then **Save As**, and choose **Word Document (.docx)**. - In **Google Docs**: **File**, then **Download**, then **Microsoft Word (.docx)**.

Before you export, check the three things that convert badly:

- **Use Word's Heading styles for section titles.** In Word, select a section title and choose **Home**, then **Heading 1** (main sections) or **Heading 2** (sub-sections). Do not just make the title bold or bigger. Pandoc only turns Heading-styled text into `#` headings. Bold text stays plain text, and your outline will be missing.
- **Remove underline and superscript formatting.** Underlined text becomes raw HTML tags

such as `<u>...</u>`, and superscripts such as `5<sup>th</sup>` do the same. Use plain text instead, or write `5th` and `^th^` by hand afterwards.

- **Put figures in the document, not as text.** Insert each picture with **Insert**, then **Pictures**. Pandoc can only save pictures that are embedded in the file.

Save the document in your Research folder, next to NSDF-Data, as `my-note.docx`. Put any separate figure files you want to keep in that folder too.

Step 2. Install pandoc once. In the terminal, with your environment active:

```
conda activate darkmatter_cli_env
conda install -c conda-forge pandoc -y
```

On macOS you can also use `brew install pandoc`. On Windows, the pandoc website has an installer.

Step 3. Convert. Go to the folder that holds the `.docx` and run:

```
pandoc my-note.docx -t gfm --extract-media=img -o my-note.md
```

This writes `my-note.md` and saves any pictures from the document into an `img` folder.

Checkpoint

Open `my-note.md` in a plain text editor. Every section title you styled as Heading 1 or Heading 2 should start with `#` or `##`. Every picture you inserted should appear as a line starting `! [`, and its file should be in the `img` folder. If you see no `#` lines at all, your titles were not Heading-styled. If the file is one long line of symbols, you saved the wrong type of file: check that it is a `.docx`, not a `.doc` or a PDF.

Careful

Look for leftover HTML. Search the file for `<u>`, `<sup>`, `<sub>` and `<span>`. Each one is formatting that did not convert. Delete the tags and keep the words, or rewrite them in plain text. A note with raw HTML tags in it will look broken on the site.

Tip

Test a conversion before you rely on it. Convert a short two-section draft with one picture first. If the headings and the picture both come through, convert the full note.

Step 4. Add the header. Open `my-note.md` and add the header block (see *The header* above) at the very top. Conversion does not add it for you.

Step 5. Check it. Go through the checklist at the end of this guide. Check every number, every heading you did use and every figure path. Figures must still exist in the `img` folder after you move the note.

Careful

Do not convert a PDF. The text of a PDF cannot be recovered reliably, so write the note again in a `.md` file, or copy the text from the PDF into your editor and check it carefully.

The header

Start your note file with a short block like this. These are the fields the site will read:

```
---
id: "TBD"
title: "Why does the 1000-sample window see more pulses?"
author: "Your Name"
date: "2026-10-05"
status: "In progress"
keywords: "baseline, pretrigger window, dump 1"
description: "One sentence: what you did and what you found."
---
```

Leave `id` as TBD. The reviewer gives your note its number (S1, S2, ...) before it is published.

Status is one of: *In progress* (you are still working on it), *Complete* (the question is answered and you are not going to change it), or *Outdated* (a later note or a fix has superseded it). New notes start as *In progress*.

Writing the body

- **Say what you did in order**, with enough detail that someone could repeat it: the file, the detector, the window length, the cut.
- **Use the project's words.** A *segment* is a stretch of consecutive events with the same trigger type. A *run* is a whole data-taking period. Do not use *run* for a segment.
- **Define each quantity once**, in the *Data and setup* section, and use the same name every time after. If you call it `bstd` in one place and “baseline noise” in another, the reader will think they are different things.
- **Write short sentences.** Put the number first, then what it means: “The median is 1.53 counts, so a typical trace is flat noise.”
- **Say how sure you are.** “The two groups look the same” is weaker than “the medians are 1.51 and 1.58, from 141 and 38 traces.” Give the counts.
- **Separate what you measured from what you think.** A sentence in *Results* states a number. A sentence in *Reading* says what it might mean.

Tip

If you change your mind, say so. Notes that show a correction are more useful than notes that pretend nothing went wrong. Put the correction in a clearly labelled box, with the date.

Figures and tables

- **Every figure needs a caption** that says what is plotted, for which data, and what to look for. “Bstd against event number, dump 1. Grey: trigger type Unknown.” is a good caption.
- **Label the axes with units.** ADC counts, microseconds, or samples. Do not leave the units out.
- **Save figures as PNG** from your notebook, at a readable size, into the note's image folder.
- **Use a table** when you are comparing numbers. Use a figure when you are showing shape.

Pinning your code

A note is only useful if the reader can find the code that made its numbers. Link to your notebook and library files at a **specific commit**, not at the master branch, because master changes:

1. Commit your notebook to your branch (see the first analysis guide).
2. Find the commit's short code, for example a1b2c3d.
3. Link to https://github.com/villano-lab/NSDF-Data/blob/a1b2c3d/R76/analysis_notes/your-

Say in the note which commit you pinned.

Outline and links

If your note has more than two or three sections, add a short outline near the top, with a link to each one. Readers use it to jump to what they need. The site builds the outline from your **##** headings, so use one **##** heading for each section. A note with no **##** headings simply has no outline.

Checking your note before you submit

Go through this list before you ask for review:

- ☐ The title is a question.
- ☐ Every number in the text can be found in your notebook output.
- ☐ Every figure has a caption with units and a sentence on what to look for.
- ☐ Every notebook link is pinned to a commit.
- ☐ The header is complete.
- ☐ Any section you use says something a reader needs. Empty or filler sections are removed.
- ☐ You have said what you did *not* check, if the note makes a claim.
- ☐ If you wrote next steps, they are concrete.
- ☐ You have read it aloud once, looking for a word a newcomer would not know.

Session Info

Guide version 5, 5 October 2026. Written for every computer. Please tell the project lead where a step was unclear.

Next

When the note is written and checked, go to **student guide 4: publishing a note**.

Where to get help

Anthony Villano, project lead: anthony.villano@ucdenver.edu

When you write, include the note's title, the step or section you are stuck on, and the sentence or number that worries you. Nobody will be annoyed by a question about a note that is not finished.

Key links

- Notes site: <https://villano-lab.github.io/NSDF-Data/>
- Repository: <https://github.com/villano-lab/NSDF-Data>
- Example of a good note with a correction box: <https://villano-lab.github.io/NSDF-Data/notes/note-02a-bstd-vs-time.html>